

INDIRA GANDHI NATIONAL OPEN UNIVERSITY, (IGNOU)



IGNOU REGIONAL CENTER, NOIDA

BY

Atif Zafar

ENROLMENT NO. - 2350200519

UNDER THE GUIDANCE OF

Mr. Faraz Ahmad

PROJECT PROPOSAL (BCSP-064)

ONLINE RENTAL MANAGEMENT SYSTEM

Submitted to the School of Computer and Information Science,
IGNOU in Partial fulfillment of the requirement for the degree of
BACHELOR'S OF COMPUTER APPLICATION (BCA)



SCHOOL OF COMPUTER AND INFORMATION SCIENCES
INDIRA GANDHI NATIONAL OPEN UNIVERSITY
PLOT NO. 88, VILLAGE-TUSYANA KNOWLEDGE PARK 5,
GREATER NOIDA, GAUTAM BUDDHA NAGAR(U.P) - 201306

Table of Contents

1. Project Overview
2. Introduction and Goals
3. Technology Classification
4. System Intent & Coverage
5. Technology Evaluation
6. System Requirements Analysis
7. System Development Life Cycle
8. Data Flow Diagrams (DFD)
9. Entity Relationship Diagram (ERD)
10. Use Case Diagram
11. Class Diagram
12. Data Structure (Corrected — 3NF Compliant)
13. Process Logic
14. Testing
15. Implementation Challenges
16. Industry / Client Interface
17. Scope of Future Application
18. Bibliography

1. PROJECT OVERVIEW

Project Title: "Online Rental Management System (ORMS)"

The Online Rental Management System (ORMS) is an innovative, web-based digital platform designed to revolutionize how individuals and businesses lend and rent products in a peer-to-peer ecosystem. The system enables any registered user to list their personal or business assets — ranging from electronics and clothing to furniture and vehicles — for rent, while other users can discover, request, and rent these products at agreed prices and durations.

The platform operates through two integrated operational frameworks:

- **Owner (Lender) Framework** – Enables users to upload and manage product listings, set rental prices and availability, approve or reject rental requests, track active rentals, and receive payments through a real-time inventory dashboard.
- **Renter (Borrower) Framework** – Allows users to search and discover available products, view detailed listings with images and pricing, submit rental requests with desired duration, make security deposits and rental payments, and manage their rentals through a personal dashboard.

ORMS creates an optimized digital marketplace that enhances the rental economy by connecting product owners with renters, reducing idle asset time, and promoting a sustainable sharing culture through modern web technology.

2. INTRODUCTION AND GOALS

2.1 Project Context

The sharing economy has emerged as one of the most transformative business models of the digital age. Platforms that connect asset owners with temporary users have proven their viability and immense market potential. However, most existing rental solutions are category-specific, leaving a significant gap for a unified, multi-category peer-to-peer rental platform.

ORMS addresses this critical gap by providing a sophisticated, general-purpose rental platform built on robust OOP and RDBMS principles. Developed as part of the BCA curriculum at IGNOU, this project applies established software engineering principles to solve a genuine marketplace challenge. The system manages complete rental workflows including user onboarding, product listing, rental requests, transaction processing, security deposits, fine management, and review systems — all through a centralized, secure database architecture.

2.2 Objectives

The primary strategic objectives of ORMS are:

- To develop a cost-effective and reliable peer-to-peer rental marketplace utilizing OOP and RDBMS principles with PHP and MySQL.
- To enable any user to act as both Owner (lender) and Renter (borrower) within the same platform account using a normalized role management approach.
- To provide a comprehensive product listing module supporting multiple categories such as Electronics, Clothing, Furniture, Vehicles, and Tools.
- To implement a secure rental request and approval workflow with real-time product availability tracking using ACID-compliant database transactions.
- To build a transparent financial management system covering rental charges, security deposits, refunds, and damage fines with defined calculation formulas.
- To ensure secure data management with role-based access control, bcrypt password hashing, and robust input validation.
- To facilitate communication between Owners and Renters through a database-driven notification and dispute resolution system.
- To provide an intuitive, responsive user interface built with Tailwind CSS, accessible across diverse devices.
- To demonstrate practical application of software engineering concepts and SDLC practices for academic development.

3. TECHNOLOGY CLASSIFICATION

Implementation Framework

ORMS is developed as an OOP and RDBMS-based web application utilizing a contemporary technology stack selected for robust performance, responsive design, and secure application delivery.

Component	Technology / Tool
Backend Development	PHP 8.x (OOP-based with Classes, Inheritance, Interfaces)
Database Management	MySQL 8.0 (3NF Normalized Schema, ACID Compliant)
Frontend Development	HTML5, JavaScript (ES6+), CSS3, Tailwind CSS
Server Infrastructure	Apache HTTP Server 2.4
Development Environment	XAMPP (Integrated PHP + MySQL + Apache)
Security Framework	bcrypt Password Hashing, Prepared Statements, CSRF Tokens, <code>finfo_file()</code> MIME Validation
File Uploads	PHP File Handling with magic-byte MIME verification
Session Management	PHP Sessions with <code>session_regenerate_id()</code> — Fixation Prevention
Notification System	Database-driven NOTIFICATION table with polling

4. SYSTEM INTENT & COVERAGE

4.1 Purpose

ORMS aims to efficiently manage a complete peer-to-peer rental marketplace through a secure, multi-role web platform. The system streamlines essential processes including user registration, product listing management, rental request workflows, transaction processing, security deposit handling, fine calculation for damages and late returns, and a review & rating system — managed through a fully normalized relational database with referential integrity.

4.2 Scope

The system encompasses digital management of all rental operations. It manages records including user profiles, product catalogs with multi-category and multi-image support, rental request history, transaction records, security deposits, fine records, dispute records, notifications, and review data — all through a well-normalized relational database architecture (3NF compliant) designed for optimal performance and scalability.

As a web-based solution accessible via modern browsers, four user roles are supported:

Role	Access Level	Key Permissions
Guest	Public (Read Only)	Browse catalog, view product details, view public reviews
Renter	Authenticated	Search & filter, submit requests, make payments, track rentals, submit reviews
Owner	Authenticated	Upload & manage listings, approve/reject requests, track earnings, raise fines
Admin	Full Access	Manage all users, resolve disputes, view reports, configure system, manage categories

5. TECHNOLOGY EVALUATION

For implementing ORMS, several technologies were evaluated based on project requirements, learning objectives, resource constraints, and suitability for a peer-to-peer marketplace.

Component	Selected Technology	Justification
Backend Development	PHP 8.x	Extensive documentation, strong OOP support, ideal for web applications
Hosting Environment	Apache 2.4	Reliability, wide compatibility, integrated in XAMPP stack
Data Storage	MySQL 8.0	ACID compliance, strong relational integrity, ideal for multi-table rental data
Client-Side Processing	JavaScript ES6+	Browser compatibility, dynamic UI updates, form validation, AJAX calls
Styling & Layout	Tailwind CSS	Utility-first responsive design, rapid UI development, mobile-first approach
Development Env.	XAMPP	Integrated PHP, MySQL, Apache for streamlined local development
Password Security	bcrypt Hashing	Industry-standard adaptive hashing — resistant to brute-force attacks
SQL Injection Prevention	Prepared Statements	Parameterized queries prevent all SQL injection vectors
File MIME Verification	finfo_file() PHP	Server-side magic-byte verification — cannot be spoofed unlike Content-Type header
Session Security	session_regenerate_id()	Prevents session fixation attacks on login/privilege change

6. SYSTEM REQUIREMENTS ANALYSIS

6.1 Problem Definition

ORMS addresses critical challenges in the rental marketplace through a sophisticated web-based application. Traditional rental processes are fragmented, manual, and trust-deficient, making it difficult for individuals to rent their idle assets or find trustworthy rental services. The platform provides comprehensive digital management of all rental lifecycle stages — from product discovery and rental requests to payments, returns, and dispute resolution.

Identified Operational Areas:

- Administrative Control (ADMIN table)
- User Management — Owners and Renters (USER + USER_ROLES tables)
- Product Catalog Management (PRODUCT + PRODUCT_IMAGES tables)
- Product Category Management (CATEGORY table)
- Rental Request & Approval (RENTAL_REQUEST table)
- Transaction & Payment Management (TRANSACTION table)
- Fine & Damage Management (FINE table)
- Review & Rating System (REVIEW table)
- Notification Management (NOTIFICATION table) — New
- Dispute Management (DISPUTE table) — New

6.2 Functional Specifications

- System Administration – Comprehensive platform oversight, dispute resolution, fine configuration, and security controls.
- User Registration & Authentication – Dual-role user accounts (via USER_ROLES table) supporting both Owner and Renter activity; secure login with bcrypt and session management; password reset via email token.
- Product Listing Management – Multi-category product uploads with separate image table (PRODUCT_IMAGES), pricing, deposit amount, condition details, and availability control.
- Search & Discovery – Advanced filtering by category, location, price range, and availability status.
- Rental Request Module – Request submission, approval/rejection workflows, cancellation policy (with deposit refund rules), and status tracking.
- Financial Management – Security deposit handling, rental charge calculation (computed at application layer, not stored), refund processing, and fine deduction from deposit.
- Fine Management – Configurable per-day late return fine (set by Admin), damage fine raised by Owner, fine deducted from security deposit.
- Review & Rating System – Post-rental reviews for products; enforced via rental completion check.
- Notification System – Database-driven notifications for all key events (request updates, payment confirmations, due date reminders).
- Dispute Management – Structured dispute filing and Admin resolution workflow.

6.3 Hardware & Software Requirements

System Specifications:

Component	Minimum Requirement	Recommended Requirement
Processor	Intel Core i3	Intel Core i5 8th Generation or higher
RAM	4 GB	8 GB DDR4
Storage	250 GB HDD	500 GB SSD
Display	1366 × 768	1920 × 1080
Network	5 Mbps broadband	10 Mbps dedicated connection
Operating System	Windows 10	Windows 11 / Linux Ubuntu 20.04 LTS
Database	MySQL 5.7	MySQL 8.0
Web Server	Apache 2.4	Apache 2.4 with SSL (HTTPS)
Browser	Chrome 90+	Latest Chrome / Firefox / Edge
Backup	Manual backup system	Automated daily backup with offsite copy
PHP Version	PHP 7.4	PHP 8.1+ (typed properties, named arguments)

6.4 Module Description

ORMS implements ten core modules with precise functionality and seamless integration:

1. System Administration Module – Platform configuration, user management, dispute resolution, and reporting.
2. User Registration & Authentication Module – Account creation, login, role assignment via USER_ROLES table, session management, and password reset.
3. Product Listing Module – Product upload with multiple images (stored in PRODUCT_IMAGES table), editing, category assignment, and availability toggle.
4. Search & Discovery Module – Advanced search, filtering by category/price/location, and product detail viewing.
5. Rental Request Module – Request submission, owner approval/rejection, cancellation workflow, status tracking, and rental period management.
6. Financial Management Module – Security deposit collection, rent calculation (computed dynamically), refund processing with fine deduction, and payment history.
7. Fine Management Module – Late return detection, configurable per-day fine calculation (Admin-set rate), damage fine by Owner, fine payment tracking.
8. Review & Rating Module – Post-rental reviews enforced by rental completion status check.
9. Notification Module – Database-driven NOTIFICATION table records all system events; polling-based delivery to users.
10. Dispute Management Module – Structured dispute filing, Admin resolution workflow, and audit trail.

7. SYSTEM DEVELOPMENT LIFE CYCLE

7.1 Development Methodology

The System Development Life Cycle (SDLC) provides a structured framework for planning, designing, building, testing, and deploying software systems. Common SDLC models include:

- Waterfall Model
- Prototype Model
- Spiral Model
- Incremental Development Model
- Agile / Scrum Model

For ORMS, the Incremental Development Model was adopted. This model divides development into planned increments, where each increment delivers a working subset of functionality. It combines the disciplined planning of Waterfall with iterative refinement, allowing for continuous feedback and modular quality assurance.

The development was executed through the following phases:

Phase	Activities	Deliverables
Requirements & Design	Analysis of rental marketplace workflows; UML diagram creation (ERD, DFD, Class Diagram, Use Case); database schema normalization to 3NF	SRS Document, UML Diagrams, Normalized Schema
Increment 1 — Database & Auth	Normalized database schema setup; User Registration/Login; Role-based access via USER_ROLES table; Session management with fixation prevention	Functional auth system
Increment 2 — Product Module	Product listing with PRODUCT_IMAGES table; category management; image upload with finfo_file() MIME validation	Product listing module
Increment 3 — Rental Workflow	Rental request submission; Owner approval/rejection; availability locking via SELECT FOR UPDATE; cancellation policy implementation	Rental request module
Increment 4 — Financial Module	Security deposit collection; dynamic rent calculation; refund processing; fine calculation with Admin-configurable rate	Financial + Fine modules
Increment 5 — Notifications & Disputes	NOTIFICATION table implementation; DISPUTE workflow; Admin oversight dashboard	Notification + Dispute modules
Integration & Testing	Unit, Integration, System, Security, and Acceptance testing across all modules	Test Reports, Bug Fixes
Deployment & Validation	Controlled environment deployment; User Acceptance Testing; feedback incorporation	Deployed system, UAT Report
Documentation & Maintenance	User guides, admin documentation, backup procedures	Complete project documentation

7.2 System Architecture

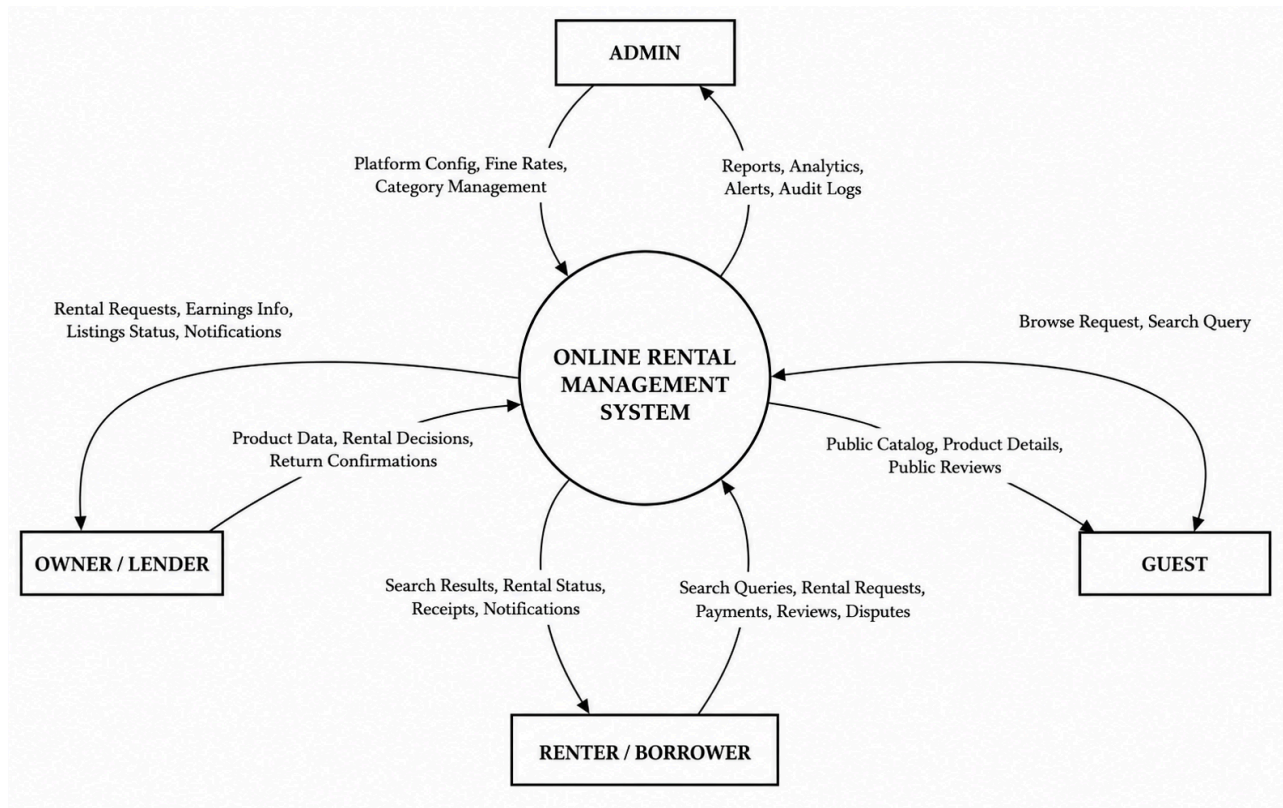
The system is built on a classic Three-Tier Architecture:

- **Presentation Layer:** HTML5, Tailwind CSS, JavaScript ES6+. Fully responsive design using Tailwind's utility-first breakpoint system. Dynamic UI updates via AJAX calls without full page reloads.
- **Application Layer:** PHP 8.x using Object-Oriented Programming (OOP) principles. Handles input validation, session management, role-based access control, and all business logic including dynamic rent calculation, fine computation, and deposit refund logic.
- **Data Layer:** MySQL 8.0 relational database. Schema normalized to Third Normal Form (3NF). Foreign key constraints enforce referential integrity. ACID compliance guarantees transaction reliability. SELECT FOR UPDATE used for concurrent rental request locking.

8. DATA FLOW DIAGRAMS (DFD)

8.1 Level 0 DFD (Context Level)

The Context DFD shows the entire ORMS as a single process with four external entities — Admin, Owner, Renter, and Guest — and their bidirectional data flows with the system.

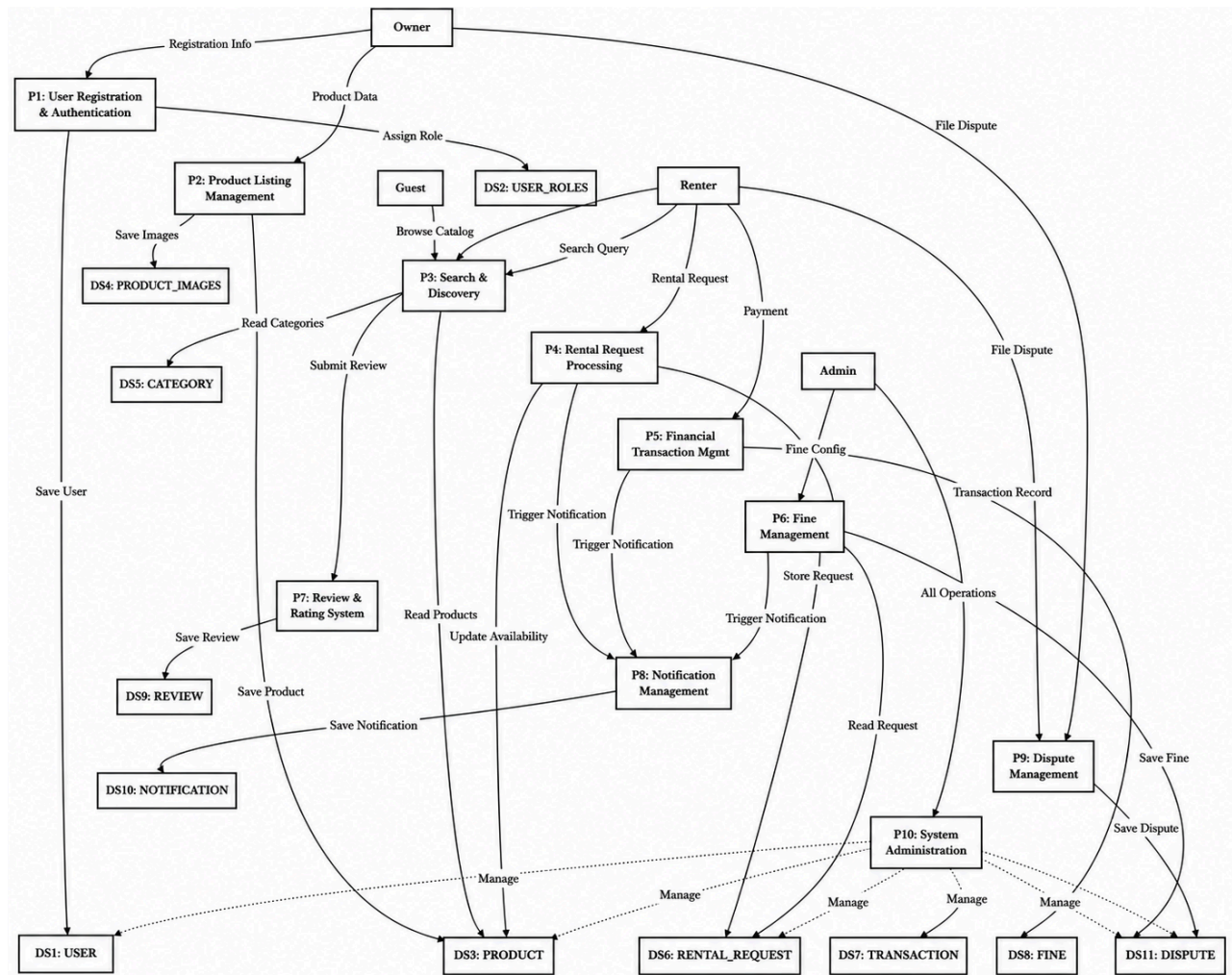


External entities and their data flows:

External Entity	Data INTO System	Data FROM System
Admin	Platform config, fine rates, dispute resolutions, category management	Reports, alerts, analytics, audit logs
Owner / Lender	Product data, rental decisions (approve/reject), return confirmations, fine reports	Rental requests, earnings info, listings status, notifications
Renter / Borrower	Search queries, rental requests, payments, review submissions, dispute filings	Search results, listings, rental status, receipts, notifications
Guest	Browse requests, search queries	Public product catalog, product details, public reviews

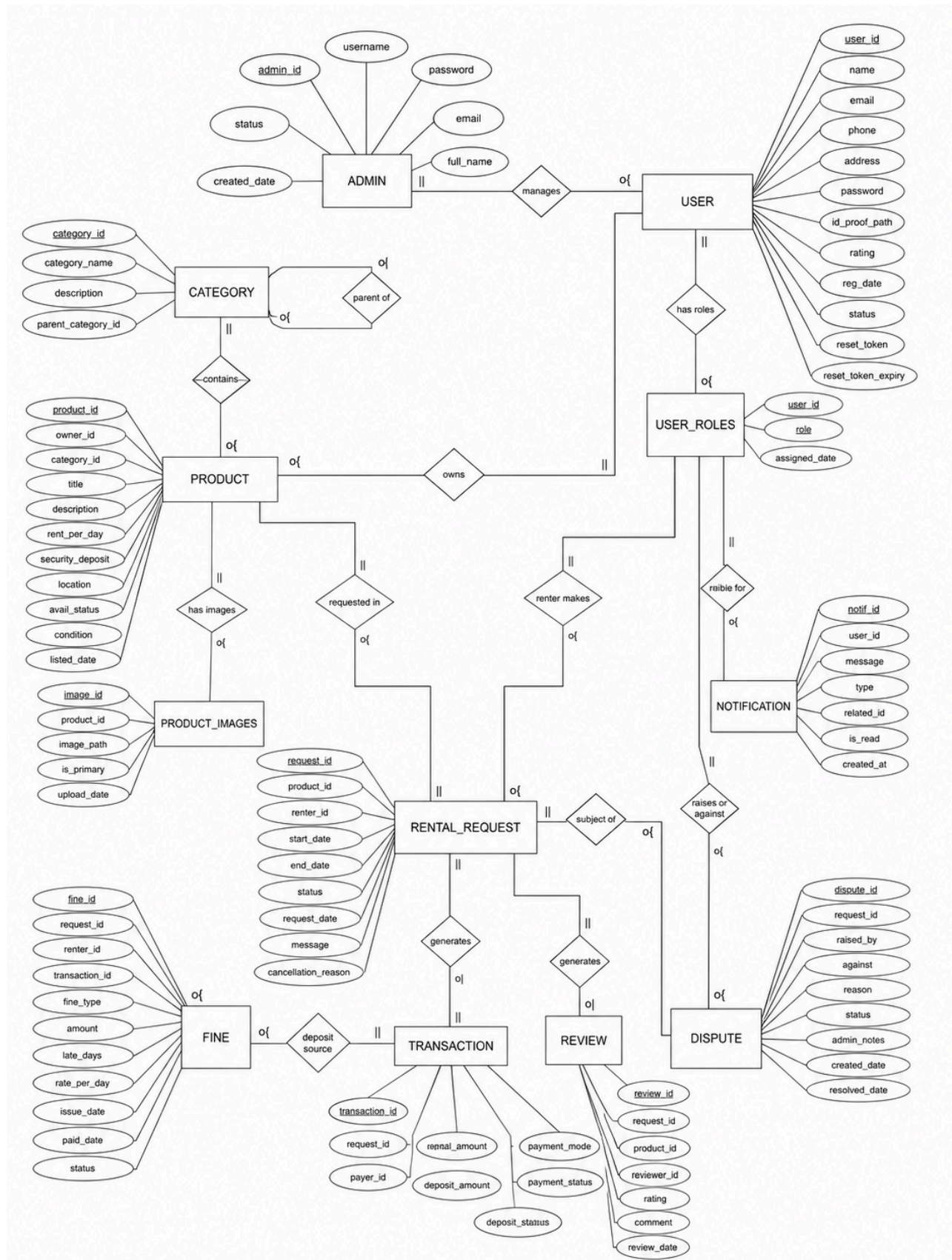
8.2 Level 1 DFD

Decomposes the system into ten major processes and their interconnecting data stores.



Process	Process Name	Key Data Stores Used
P1	User Registration & Authentication	DS1: USER, DS9: USER_ROLES
P2	Product Listing Management	DS2: PRODUCT, DS8: PRODUCT_IMAGES, DS3: CATEGORY
P3	Search & Discovery	DS2: PRODUCT, DS3: CATEGORY
P4	Rental Request Processing	DS4: RENTAL_REQUEST, DS2: PRODUCT
P5	Financial Transaction Management	DS5: TRANSACTION, DS4: RENTAL_REQUEST
P6	Fine Management	DS6: FINE, DS4: RENTAL_REQUEST, DS5: TRANSACTION
P7	Review & Rating System	DS7: REVIEW, DS4: RENTAL_REQUEST
P8	Notification Management	DS10: NOTIFICATION
P9	Dispute Management	DS11: DISPUTE, DS4: RENTAL_REQUEST

9. ENTITY RELATIONSHIP DIAGRAM (ERD)

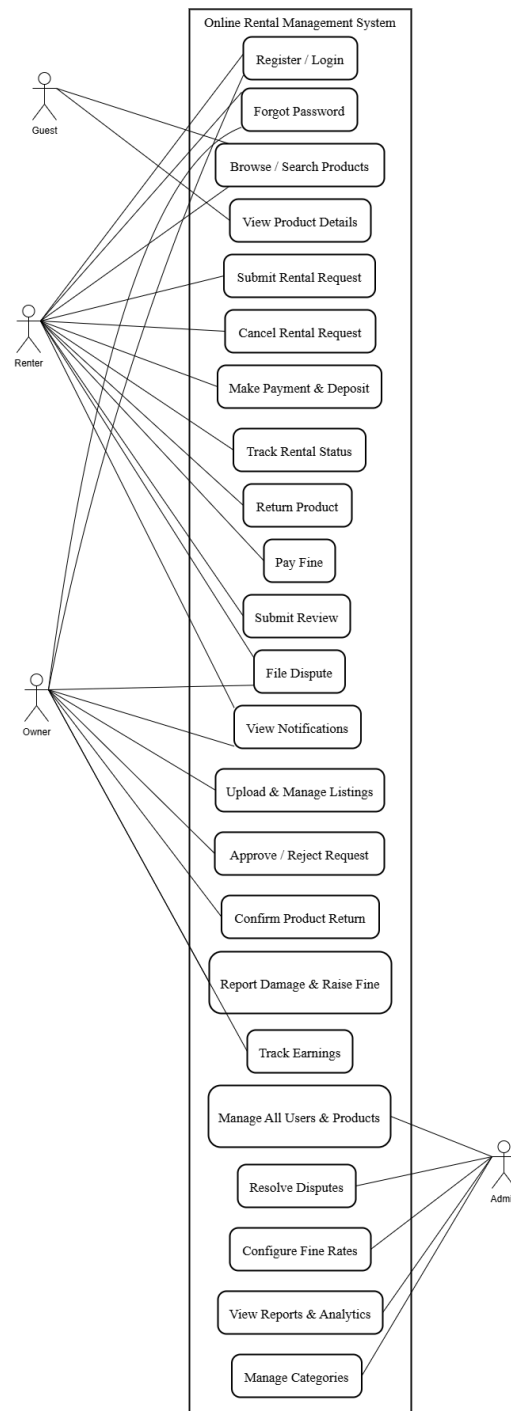


9.1 Entity Descriptions

Entity	Description	Key Relationships
ADMIN	Platform administrator with full oversight permissions	MANAGES → USER (1:M)
USER	Represents both Owners and Renters in one account	Has roles via USER_ROLES (1:M)
USER_ROLES	Junction table for multi-role support (replaces ENUM 'Both')	USER (M:1), role ENUM(Owner, Renter)
CATEGORY	Product categories (Electronics, Clothing, etc.)	CONTAINS → PRODUCT (1:M)
PRODUCT	Listed by Owners, belongs to a Category	OWNS (USER 1:M), CONTAINS (CATEGORY 1:M)
PRODUCT_IMAGES	Separate image table — fixes 1NF violation of comma-separated images	PRODUCT (M:1) — 0..* images per product
RENTAL_REQUEST	Rental request from Renter for a Product	USER (M:1), PRODUCT (M:1) — owner_id derived via JOIN
TRANSACTION	Payment record generated on request approval	RENTAL_REQUEST (1:1)
FINE	Generated for late return or damage	RENTAL_REQUEST (M:1), USER/renter (M:1)
REVIEW	Post-rental review; enforced via rental completion check	PRODUCT (M:1), USER/reviewer (M:1), RENTAL_REQUEST (1:1)
NOTIFICATION	System event notifications for all users	USER (M:1)
DISPUTE	Structured dispute record for Admin resolution	RENTAL_REQUEST (M:1), USER raised_by + against

10. USE CASE DIAGRAM

The Use Case Diagram identifies four actors and their interactions. Corrections from original: added Cancel Rental Request, Forgot Password, View Earnings Report, File Dispute, and View Notifications use cases.

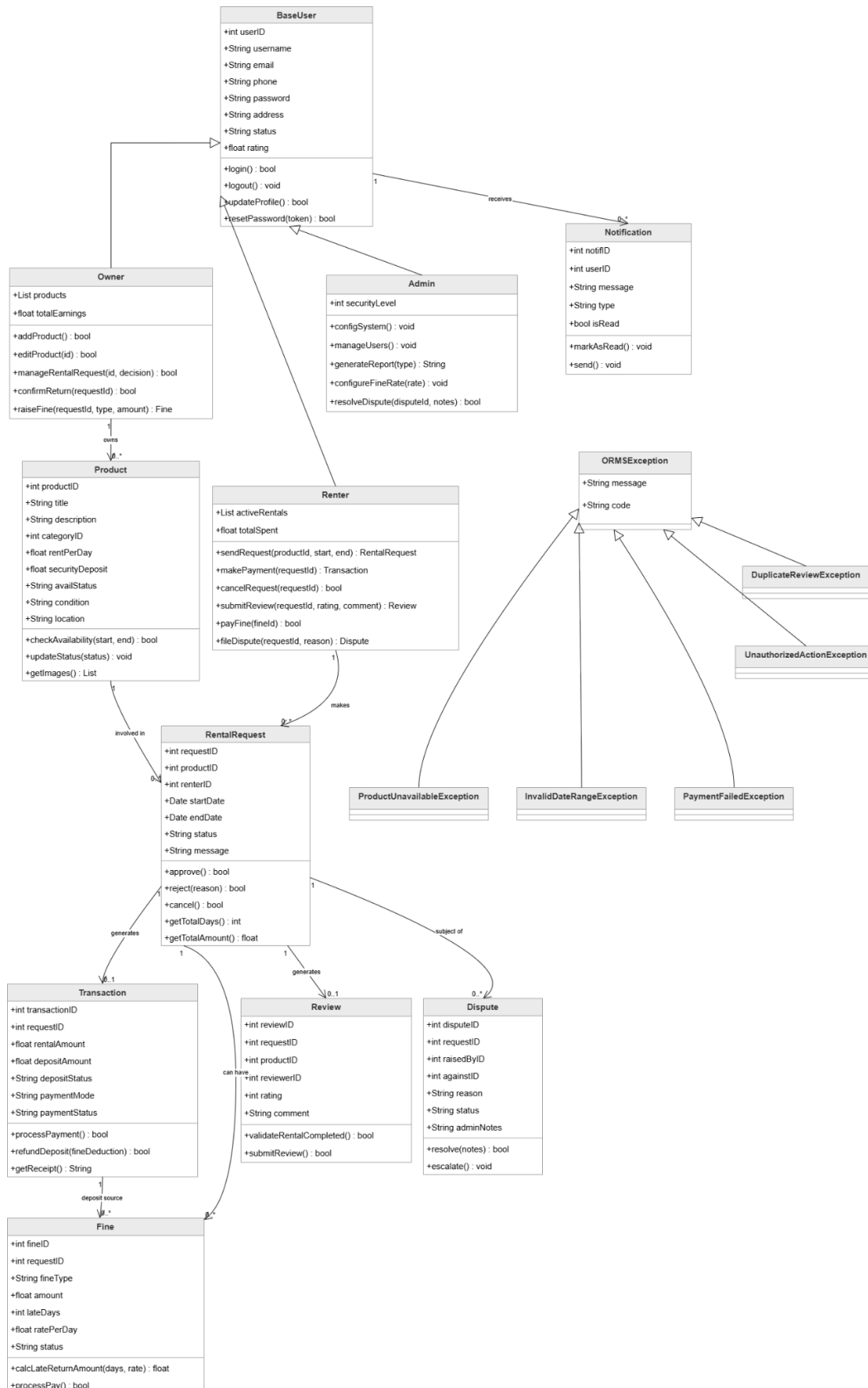


Actor	Use Cases
Guest	Browse Products, Search by Category, View Product Details, View Public Reviews
Renter	Register / Login, Forgot Password, Search & Filter Products, Submit Rental Request, Cancel Rental Request (before approval), Make Payment & Deposit, Track Rental Status, Return Product, Pay Fine, Submit Review, File Dispute, View Notifications

Actor	Use Cases
Owner	Register / Login, Forgot Password, Upload & Manage Product Listings, Approve / Reject Rental Request, Confirm Product Return, Report Damage & Raise Fine, Track Earnings, View Notifications, Respond to Dispute
Admin	Manage All Users & Products, Resolve Disputes, Configure Fine Rates, View Reports & Analytics, Configure System Settings, Manage Categories, Override Decisions

11. CLASS DIAGRAM

The corrected Class Diagram adds missing exception classes, fixes the orphan maxListings attribute, and adds proper validation methods. All attributes are consistent with the database schema.



11.1 Primary Classes

Class	Attributes	Methods
BaseUser (Abstract)	userID, username, email, phone, password, address, status, rating	login(): bool, logout(): void, updateProfile(): bool, resetPassword(token): bool
Owner extends BaseUser	products: List<Product>, totalEarnings: float	addProduct(): bool, editProduct(id): bool, manageRentalRequest(id, decision): bool, confirmReturn(requestId): bool, raiseFine(requestId, type, amount): Fine
Renter extends BaseUser	activeRentals: List<RentalRequest>, totalSpent: float	sendRequest(productId, start, end): RentalRequest, makePayment(requestId): Transaction, cancelRequest(requestId): bool, submitReview(requestId, rating, comment): Review, payFine(fineId): bool, fileDispute(requestId, reason): Dispute
Admin extends BaseUser	securityLevel: int	configSystem(): void, manageUsers(): void, generateReport(type): Report, configureFineRate(rate): void, resolveDispute(disputeId, notes): bool
Product	productId, title, description, categoryID, rentPerDay, securityDeposit, availStatus, condition, location, listedDate	checkAvailability(start, end): bool, updateStatus(status): void, getImages(): List<ProductImage>
RentalRequest	requestID, productId, renterID, startDate, endDate, status, message, requestDate	approve(): bool, reject(reason): bool, cancel(): bool, getTotalDays(): int, getTotalAmount(): float [computed, not stored]
Transaction	transactionID, requestID, payerID, amount, depositAmount, paymentMode, paymentDate, paymentStatus	processPayment(): bool, refundDeposit(fineDeduction: float): bool, getReceipt(): string
Fine	fineID, requestID, renterID, fineType, amount, issueDate, paidDate, status	calcLateReturnAmount(days: int, ratePerDay: float): float, processPay(): bool
Review	reviewID, requestID, productId, reviewerID, rating, comment, reviewDate	validateRentalCompleted(): bool, submitReview(): bool
Notification	notifID, userID, message, type, isRead, createdAt	markAsRead(): void, send(): void
Dispute	disputeID, requestID, raisedByID, againstID, reason, status, adminNotes, createdAt	resolve(adminNotes): bool, escalate(): void

11.2 Exception Classes

Exception Class	Thrown When
ORMSException (Base)	Base exception for all ORMS-specific errors
ProductUnavailableException	Rental request submitted for already-rented product
InvalidDateRangeException	start_date >= end_date, or past dates provided
PaymentFailedException	Transaction processing fails
UnauthorizedActionException	Role-based access violation detected
DuplicateReviewException	Renter tries to submit second review for same rental

12. DATA STRUCTURE (CORRECTED — 3NF COMPLIANT)

12.1 Database Normalization (3NF Compliance)

- No redundant data — each non-key attribute depends only on the primary key.
- Referential integrity maintained through foreign key constraints across all related tables.
- Atomic values in all fields — no comma-separated multi-values (PRODUCT_IMAGES table used instead).
- No transitive dependencies — owner_id derivable via JOIN, total_days and total_amount computed at runtime.
- Multi-role support via USER_ROLES junction table — no composite ENUM values.

12.2 ADMIN Table

Field Name	Data Type	Constraints	Description
admin_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique administrator identifier
username	VARCHAR(50)	UNIQUE, NOT NULL	Login username
password	VARCHAR(255)	NOT NULL	bcrypt hashed password
email	VARCHAR(100)	UNIQUE, NOT NULL	Contact email address
full_name	VARCHAR(100)	NOT NULL	Complete administrator name
status	ENUM('Active','Inactive')	DEFAULT 'Active'	Account status
created_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp

12.3 USER Table

Field Name	Data Type	Constraints	Description
user_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique user identifier
name	VARCHAR(100)	NOT NULL	User full name
email	VARCHAR(100)	UNIQUE, NOT NULL	Contact email — used for login
phone	VARCHAR(15)	NOT NULL	Contact phone number
address	TEXT	NOT NULL	Physical address
password	VARCHAR(255)	NOT NULL	bcrypt hashed password
id_proof_path	VARCHAR(255)	NOT NULL	Randomized server path to ID document

Field Name	Data Type	Constraints	Description
rating	DECIMAL(3,2)	DEFAULT 0.00	Average user rating (computed from REVIEW)
reg_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Registration timestamp
status	ENUM('Active','Inactive','Banned')	DEFAULT 'Active'	Account status
reset_token	VARCHAR(64)	NULL	Password reset token (hashed)
reset_token_expiry	DATETIME	NULL	Token expiry — null if no active reset

12.4 USER_ROLES Table (NEW — Replaces ENUM Anti-Pattern)

Field Name	Data Type	Constraints	Description
user_id	INT	PK, FK → USER(user_id)	Reference to user
role	ENUM('Owner','Renter')	PK (composite)	Role assigned to user
assigned_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	When this role was assigned

12.5 CATEGORY Table

Field Name	Data Type	Constraints	Description
category_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique category identifier
category_name	VARCHAR(100)	UNIQUE, NOT NULL	Category name (e.g., Electronics)
description	TEXT	NULL	Optional category description
parent_category_id	INT	FK → CATEGORY(category_id), NULL	Self-reference for subcategories (NULL = top-level)

12.6 PRODUCT Table (Corrected)

Field Name	Data Type	Constraints	Description
product_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique product identifier

Field Name	Data Type	Constraints	Description
owner_id	INT	FK → USER(user_id), NOT NULL	Owner of the product
category_id	INT	FK → CATEGORY(category_id), NOT NULL	Product category
title	VARCHAR(255)	NOT NULL	Product listing title
description	TEXT	NOT NULL	Detailed product description
rent_per_day	DECIMAL(10,2)	NOT NULL, CHECK > 0	Daily rental charge (INR)
security_deposit	DECIMAL(10,2)	NOT NULL, CHECK >= 0	Refundable security deposit (INR)
location	VARCHAR(100)	NOT NULL	Product pickup/delivery location
avail_status	ENUM('Available', 'Rented', 'Unavailable')	DEFAULT 'Available'	Current availability status
condition	ENUM('New', 'Good', 'Fair', 'Poor')	NOT NULL	Product condition
listed_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date product was listed

12.7 PRODUCT_IMAGES Table (NEW — Fixes 1NF Violation)

Field Name	Data Type	Constraints	Description
image_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique image identifier
product_id	INT	FK → PRODUCT(product_id), NOT NULL	Reference to product
image_path	VARCHAR(255)	NOT NULL	Randomized server path to image file
is_primary	BOOLEAN	DEFAULT FALSE	Whether this is the main display image
upload_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Upload timestamp

12.8 RENTAL_REQUEST Table (Corrected)

Field Name	Data Type	Constraints	Description
request_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique request identifier

Field Name	Data Type	Constraints	Description
product_id	INT	FK → PRODUCT(product_id), NOT NULL	Requested product
renter_id	INT	FK → USER(user_id), NOT NULL	Renter making the request
start_date	DATE	NOT NULL	Rental start date
end_date	DATE	NOT NULL, CHECK > start_date	Rental end date
status	ENUM('Pending','Approved','Active','Completed','Rejected','Cancelled')	DEFAULT 'Pending'	Current rental status
request_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date request was submitted
message	TEXT	NULL	Optional message from renter to owner
cancellation_reason	TEXT	NULL	Populated if status = Cancelled

12.9 TRANSACTION Table

Field Name	Data Type	Constraints	Description
transaction_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique transaction identifier
request_id	INT	FK → RENTAL_REQUEST(request_id), NOT NULL	Related rental request
payer_id	INT	FK → USER(user_id), NOT NULL	Renter making the payment
rental_amount	DECIMAL(10,2)	NOT NULL	Computed rental amount (days × rate) stored at transaction time
deposit_amount	DECIMAL(10,2)	NOT NULL	Security deposit collected
deposit_status	ENUM('Held','Refunded','Partially Refunded','Forfeited')	DEFAULT 'Held'	Deposit disposition status
payment_mode	VARCHAR(30)	NOT NULL	Cash / UPI / Card / Bank Transfer
payment_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date and time of payment

Field Name	Data Type	Constraints	Description
payment_status	ENUM('Pending','Completed','Failed','Refunded')	DEFAULT 'Pending'	Payment processing status

NOTE: *rental_amount IS stored in TRANSACTION (not RENTAL_REQUEST) as a snapshot of the amount at the time of payment. This is correct — it serves as an immutable financial record, unlike the redundant storage in RENTAL_REQUEST.*

12.10 FINE Table

Field Name	Data Type	Constraints	Description
fine_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique fine identifier
request_id	INT	FK → RENTAL_REQUEST(request_id), NOT NULL	Related rental request
renter_id	INT	FK → USER(user_id), NOT NULL	Renter liable for the fine
transaction_id	INT	FK → TRANSACTION(transaction_id), NOT NULL	Source transaction for deposit deduction
fine_type	ENUM('Late_Return','Damage','Lost')	NOT NULL	Type of fine
amount	DECIMAL(10,2)	NOT NULL, CHECK > 0	Fine amount due (INR)
late_days	INT	NULL	Populated only for Late_Return fines
rate_per_day	DECIMAL(10,2)	NULL	Admin-configured rate used at time of fine — snapshot
issue_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date fine was issued
paid_date	DATETIME	NULL	Date fine was paid (NULL if unpaid)
status	ENUM('Unpaid','Paid','Waived','Deducted_From_Deposit')	DEFAULT 'Unpaid'	Fine payment status

Fine Formula (Late Return): *Fine Amount = late_days × rate_per_day. Where late_days = DATEDIFF(actual_return_date, end_date) and rate_per_day is the Admin-configured value (stored as snapshot in this record). Maximum fine capped at 2 × security_deposit.*

12.11 REVIEW Table

Field Name	Data Type	Constraints	Description
review_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique review identifier
request_id	INT	FK → RENTAL_REQUEST(request_id), UNIQUE	Enforces one review per rental — UNIQUE constraint
product_id	INT	FK → PRODUCT(product_id), NOT NULL	Reviewed product
reviewer_id	INT	FK → USER(user_id), NOT NULL	Renter submitting review
rating	TINYINT	NOT NULL, CHECK BETWEEN 1 AND 5	Numeric rating (1–5 stars)
comment	TEXT	NULL	Optional text review
review_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date review was submitted

12.12 NOTIFICATION Table (NEW)

Field Name	Data Type	Constraints	Description
notif_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique notification identifier
user_id	INT	FK → USER(user_id), NOT NULL	Recipient user
message	TEXT	NOT NULL	Notification message text
type	ENUM('Rental', 'Payment', 'Fine', 'Dispute', 'System')	NOT NULL	Notification category
related_id	INT	NULL	Optional reference ID (request_id, fine_id, etc.)
is_read	BOOLEAN	DEFAULT FALSE	Whether user has read this notification
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Creation timestamp

12.13 DISPUTE Table (NEW)

Field Name	Data Type	Constraints	Description
dispute_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique dispute identifier
request_id	INT	FK → RENTAL_REQUEST(req uest_id), NOT NULL	Related rental request
raised_by	INT	FK → USER(user_id), NOT NULL	User who filed the dispute
against	INT	FK → USER(user_id), NOT NULL	User the dispute is against
reason	TEXT	NOT NULL	Detailed reason for dispute
status	ENUM('Open','Un der_Review','Reso lved','Escalated')	DEFAULT 'Open'	Dispute resolution status
admin_notes	TEXT	NULL	Admin resolution notes
created_date	DATETIME	DEFAULT CURRENT_TIMESTAM P	Date dispute was filed
resolved_date	DATETIME	NULL	Date dispute was resolved

13. PROCESS LOGIC

ORMS implements comprehensive workflow management across all ten core modules:

I. System Administration Module

- Provides complete platform oversight, security controls, and dispute resolution.
- Manages global system configuration including fine rate configuration (stored in a dedicated configuration record managed via Admin panel).
- Oversees all user accounts, product listings, and financial records.
- Generates comprehensive reports on rental activity, revenue, fine collection, and platform metrics.
- Handles dispute resolution between Owners and Renters with full audit trail.

II. User Registration & Authentication Module

- New users register with personal details, ID proof upload (MIME verified via `finfo_file()`), role selection (Owner / Renter / Both via `USER_ROLES` table).
- ID proof stored with randomized filename in restricted server directory.
- Secure login with bcrypt password verification and `session_regenerate_id()` on login.
- Password reset via time-limited email token (stored hashed in `reset_token` column, 1-hour expiry).
- Role-based access enforced throughout using `USER_ROLES` table lookup on every protected page.

III. Product Listing Module

- Owners upload products with multiple images (stored in `PRODUCT_IMAGES` table — separate record per image).
- Each uploaded image validated server-side: `finfo_file()` checks actual magic bytes, not just Content-Type header.
- Allowed types: JPEG, PNG, WEBP. Maximum size: 5MB per image. Filename randomized (`uniqid() + extension`).
- Product availability auto-updated to 'Rented' when rental enters 'Active' status; reverts to 'Available' on completion or cancellation.
- Full CRUD operations with soft-delete option (status = 'Unavailable' instead of hard delete to preserve rental history).

IV. Search & Discovery Module

- Advanced search by product title, description, category, and keyword (MySQL FULLTEXT index on title, description).
- Real-time filtering by availability, price range, location, and category.
- Product detail page shows primary image (`PRODUCT_IMAGES` where `is_primary = TRUE`), all images, rental pricing, deposit info, owner rating, and existing reviews.
- Public catalog accessible to Guest users without registration.

V. Rental Request Module

- Renter submits request with `product_id`, `start_date`, `end_date` (validated: `start < end`, `start >= today`).
- Availability check: system verifies no Active/Approved request overlaps with requested dates.

- Concurrent locking: SELECT ... FOR UPDATE on PRODUCT row during availability check + status update (prevents race condition when two renters request simultaneously).
- Owner notified (NOTIFICATION record created) on new request.
- Owner approves/rejects — status transitions: Pending → Approved/Rejected.
- Cancellation Policy: Renter can cancel in Pending status — no charge. After Approved status, cancellation within 24 hours: full deposit refund. After 24 hours: deposit forfeited (deposit_status = 'Forfeited').
- Upon approval: product avail_status = 'Rented', renter redirected to payment.
- Status lifecycle: Pending → Approved → Active (after payment) → Completed (after return confirmed).

VI. Financial Management Module

- Rental amount computed dynamically: total_days = DATEDIFF(end_date, start_date); total_amount = total_days × rent_per_day.
- At payment: rental_amount and deposit_amount stored as snapshot in TRANSACTION table.
- On clean return (Owner confirms): deposit refunded in full — deposit_status = 'Refunded'.
- If fine exists: fine amount deducted from deposit first, remainder refunded — deposit_status = 'Partially_Refunded'.
- If fine exceeds deposit: renter pays difference — deposit_status = 'Forfeited', remaining fine amount = 'Unpaid'.
- Payment receipts generated from TRANSACTION record and available in user dashboard.

VII. Fine Management Module

Fine Formula: $\text{Late Return Fine} = \text{late_days} \times \text{rate_per_day}$ | Where: $\text{late_days} = \text{DATEDIFF}(\text{actual_return_date}, \text{end_date})$ | $\text{rate_per_day} = \text{Admin-configured value (e.g., } 1.5 \times \text{daily rent)}$ | $\text{Maximum cap} = 2 \times \text{security_deposit}$

- Late return auto-detected: system compares actual return date with RENTAL_REQUEST.end_date.
- Owner reports damage: raises FINE record with fine_type = 'Damage', amount entered manually by Owner, reviewed by Admin.
- Fine deduction from deposit: processed in TRANSACTION module; transaction_id FK in FINE table links deposit source.
- Admin can waive fines (status = 'Waived') with mandatory admin_notes on DISPUTE record.
- Generates fine reports for Admin oversight.

VIII. Review & Rating Module

- submitReview() first calls validateRentalCompleted(): checks RENTAL_REQUEST.status = 'Completed' AND reviewer_id = renter_id of that request.
- UNIQUE constraint on REVIEW.request_id prevents duplicate reviews at database level.
- Product average rating = AVG(rating) from REVIEW WHERE product_id = X — computed via query, not stored.
- User trust score = AVG(rating) from REVIEW for products owned by that user — computed via query.

IX. Notification Module

Trigger Event	Recipient	Notification Type
New rental request submitted	Owner	Rental
Request Approved / Rejected	Renter	Rental
Payment Successful	Renter + Owner	Payment
Rental Due Date Reminder (1 day before)	Renter	Rental
Return Confirmed by Owner	Renter	Rental
Fine Issued	Renter	Fine
Dispute Filed	Admin + Opposing User	Dispute
Dispute Resolved	Both parties	Dispute
Deposit Refunded	Renter	Payment

X. Dispute Management Module

- Either Owner or Renter can file a dispute against the other for any Completed or Active rental.
- DISPUTE record created with raised_by, against, reason, and related request_id.
- Both parties notified via NOTIFICATION.
- Admin reviews dispute: can view full rental history, transaction records, and fine records.
- Admin resolves with mandatory admin_notes — status updated to 'Resolved'; both parties notified.
- Full audit trail maintained in DISPUTE table.

14. TESTING

Testing ensures ORMS operates correctly, securely, and efficiently when handling sensitive user data, financial transactions, security deposits, and product inventory management.

14.1 Testing Methodology Framework

Test Type	Scope	Key Test Cases
Unit Testing	Individual modules in isolation	User registration validation, fine calculation formula accuracy, date range validation, MIME type verification
Integration Testing	Module interactions	Auth → Product Listing flow, Rental Request → Transaction flow, Fine → Deposit deduction flow
System Testing	End-to-end workflows	Complete rental lifecycle: register → list → search → request → pay → return → review
Security Testing	All security controls	SQL injection via prepared statements, CSRF token validation, session fixation prevention, MIME spoofing attempt
Acceptance Testing (UAT)	Stakeholder validation	Real-world scenarios with Owner and Renter personas; feedback integration
Black-box Testing	Input/output behavior	Login with invalid credentials, submission of request for unavailable product, duplicate review submission
White-box Testing	Internal logic	Fine calculation formula verification, SELECT FOR UPDATE concurrent locking, role-based access control paths
Grey-box Testing	Module interfaces	Data flow between auth, rental request, and financial management subsystems

14.2 Specialized Evaluation Protocols

Protocol	Target	Acceptance Criteria
Volume Testing	5,000+ product listings, bulk transactions	No data corruption or performance degradation
Stress Testing	100+ concurrent user sessions	System remains responsive; no deadlocks from SELECT FOR UPDATE
Performance Testing	Product search, page load, transaction processing	Search < 1 second; page load < 2 seconds; transaction < 3 seconds
Concurrency Testing	Two simultaneous requests for same product	Only one request succeeds; other receives ProductUnavailableException
Security Testing	Authentication, SQL injection, XSS, CSRF, file upload spoofing	Zero successful attacks in controlled penetration test

Protocol	Target	Acceptance Criteria
Compatibility Testing	Chrome, Firefox, Safari, Edge; mobile + tablet + desktop	Full functionality on all supported browsers and viewports
Cancellation Policy Testing	Cancel before/after 24-hour window post-approval	Correct deposit handling per policy (refund vs. forfeit)
Fine Calculation Testing	Various late return scenarios	Amount = $\text{late_days} \times \text{rate_per_day}$; capped at $2 \times$ deposit
Backup Recovery Testing	Database backup and restore	Full data recovery from latest backup with no loss

15. IMPLEMENTATION CHALLENGES

The following challenges have been identified with strategic technical solutions for each:

I. Building Trust in a Peer-to-Peer Marketplace

Complexity:

Owners may be reluctant to lend valuable items to unknown renters, and renters may be skeptical of product quality claims.

Resolution Strategy:

- ID proof verification requirement with `finfo_file()` MIME validation during registration.
- Review and rating system (REVIEW table) post-rental — enforced by rental completion check.
- Security deposit collection before rental commencement (TRANSACTION table).
- Dispute resolution mechanism (DISPUTE table) with Admin oversight and full audit trail.

II. Concurrent Rental Request Race Condition

Complexity:

When two renters simultaneously request the same product, both may pass the availability check before either updates the status, resulting in double-booking.

Resolution Strategy:

- MySQL `SELECT ... FOR UPDATE` (Pessimistic Locking) acquires a row-level lock on the `PRODUCT` row during the availability check.
- The entire operation (check availability → update status → create request) runs in a single database transaction.
- The second concurrent request blocks until the first transaction commits, then sees the updated status and receives `ProductUnavailableException`.

III. Normalization vs. Performance Trade-off

Complexity:

Strict 3NF removes stored `total_days` and `total_amount`, requiring computation on every request display. With high traffic, this may impact performance.

Resolution Strategy:

- `rental_amount` is stored as a snapshot in `TRANSACTION` (the authoritative financial record) — available for quick retrieval without recomputation for completed transactions.
- For display purposes (listing active requests), computation is lightweight: `DATEDIFF() × rent_per_day` is a simple MySQL operation.
- MySQL `INDEX` on `RENTAL_REQUEST(product_id)`, `RENTAL_REQUEST(renter_id)` and `PRODUCT(avail_status)` ensures fast query execution.

IV. Security Deposit and Refund Management

Complexity:

Handling deposits fairly — collecting, holding during rental, then refunding or partially deducting based on fine amounts — is a complex financial workflow.

Resolution Strategy:

- TRANSACTION.deposit_status tracks deposit lifecycle: Held → Refunded / Partially_Refunded / Forfeited.
- FINE.transaction_id FK links each fine to the source deposit transaction.
- Auto-refund timeout: If Owner does not confirm return within 7 days of end_date, system auto-triggers deposit refund (cron job / scheduled task).
- Partial refund calculation: refund_amount = deposit_amount - SUM(fine amounts with status != 'Waived').

V. File Upload Security

Complexity:

Allowing users to upload product images introduces security risks including malicious file uploads (PHP shells disguised as images), storage overflows, and unauthorized access.

Resolution Strategy:

- finfo_file() reads actual file magic bytes — cannot be spoofed via Content-Type header manipulation unlike mime_content_type().
- Allowed magic bytes: JPEG (FF D8 FF), PNG (89 50 4E 47), WEBP (52 49 46 46 ... 57 45 42 50).
- Maximum file size: 5MB enforced both client-side (JS) and server-side (PHP).
- Uploaded files stored with randomized uniqid() filenames in a restricted directory outside web root; served via a PHP proxy script.

VI. Dual-Role User Experience

Complexity:

A single user acting as both Owner and Renter simultaneously requires careful UI/UX design to avoid confusion between the two dashboards.

Resolution Strategy:

- USER_ROLES table allows dynamic role assignment — a user can have both Owner and Renter roles simultaneously.
- Unified account dashboard with clearly tabbed Owner and Renter sections, role-aware navigation.
- Role switching handled via session variable — no re-login required; SESSION['active_role'] set to desired role.

VII. Password Reset Security

Complexity:

Implementing secure password reset without exposing user accounts to account takeover via predictable tokens.

Resolution Strategy:

- Password reset token generated using random_bytes(32) → bin2hex() — cryptographically random.
- Token stored HASHED (password_hash()) in USER.reset_token — plain token only in the email link.
- Token expires after 1 hour (USER.reset_token_expiry); expired tokens rejected.
- Token invalidated immediately after use (set to NULL).

16. INDUSTRY / CLIENT INTERFACE

Are you doing this project for any Industry/Client?

Yes ☐ No ☒

This is an academic project developed as part of the BCA curriculum (BCSP-064) at IGNOU. It is not developed for any specific industry or client.

17. SCOPE OF FUTURE APPLICATION

ORMS has the potential to evolve into a comprehensive digital rental marketplace. Built on OOP and RDBMS principles with a PHP/MySQL/Tailwind architecture, the system provides a solid foundation for extensive future expansion.

17.1 Target Beneficiaries

- Individual consumers for peer-to-peer sharing of household items, clothing, and electronics.
- Small and Medium Enterprises (SMEs) for equipment and machinery rental management.
- Event Management Companies for costume, decor, and AV equipment rentals.
- Educational Institutions for textbook, lab equipment, and sports gear lending programs.
- Corporate Organizations for internal asset sharing and inter-department resource lending.

17.2 Immediate Practical Enhancements (6–12 months)

Enhancement	Technology	Benefit
Digital Payment Gateway	Razorpay / Stripe / UPI	Automated rental and deposit payments — eliminates manual payment tracking
SMS & Email Notifications	Twilio (SMS) + PHPMailer / SMTP	Real-time alerts replace polling-based NOTIFICATION table for critical events
Real-time Chat	WebSocket (Ratchet PHP) or Firebase	Direct Owner-Renter negotiation within platform
Geolocation Search	Google Maps JavaScript API	Show products within X km radius of renter's location
Advanced Admin Analytics	Chart.js / D3.js dashboards	Visual reports on revenue, popular categories, user activity trends

17.3 Medium-term Deployable Solutions (1–2 years)

Enhancement	Technology	Benefit
Mobile Application	React Native (Android + iOS)	On-the-go product discovery and rental management
Cloud Migration	AWS EC2 + RDS + S3 / Azure	Scalability, CDN-based image delivery, 99.9% uptime SLA
AI Pricing Suggestions	Python ML model + REST API	Recommends optimal rental prices based on demand and comparable listings
Rental Insurance Integration	Third-party insurance API	Optional rental insurance for Renters at checkout
HTTPS + SSL Enforcement	Let's Encrypt SSL	Encrypted data in transit — mandatory for payment processing

17.4 Advanced Implementation Roadmap (2–3 years)

Enhancement	Technology	Benefit
ML-based Fraud Detection	Scikit-learn / TensorFlow	Identifies suspicious rental patterns, fake listings, payment fraud
Blockchain Smart Contracts	Ethereum / Polygon + Solidity	Self-executing rental agreements — automatic deposit handling without intermediary
RFID / QR Code Tracking	Hardware + PHP QR API	Physical product condition verification at pickup and return
Predictive Analytics	Python Pandas + ML regression	Demand forecasts, optimal listing times, pricing trends for Owners
Multi-language Support	PHP i18n libraries	Expand market reach beyond English-speaking users

18. BIBLIOGRAPHY

1. Project Guide Booklet, BCSP-064, IGNOU.
2. Programming PHP by Rasmus Lerdorf, Kevin Tatroe, Peter MacIntyre — O'Reilly Media.
3. PHP & MySQL: The Missing Manual by Brett McLaughlin — O'Reilly Media.
4. Introduction to Software Engineering — IGNOU Study Material.
5. Effective Methods for Software Engineering by Pankaj Jalote — Narosa Publishing House.
6. Software Engineering by Ian Sommerville — Pearson Education, 10th Edition.
7. Fundamentals of Software Engineering by Rajib Mall — PHI Learning.
8. Database System Concepts by Silberschatz, Korth, Sudarshan — McGraw-Hill, 7th Edition.
9. PHP Official Documentation — <https://www.php.net/manual/en/index.php>
10. MySQL 8.0 Reference Manual — <https://dev.mysql.com/doc/refman/8.0/en/>
11. OWASP PHP Security Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/PHP_Configuration_Cheat_Sheet.html
12. Tailwind CSS Documentation — <https://tailwindcss.com/docs>
13. GeeksforGeeks — Database Normalization — <https://www.geeksforgeeks.org/normal-forms-in-dbms/>
14. TutorialsPoint PHP Tutorial — <https://www.tutorialspoint.com/php/index.htm>